SESSION 3

Evaluation & Iteration — The Pivot

3

Turn prompting from vibes into engineering: measure, judge, and catch regressions.

Vendredi 18 septembre · 8h00–10h30 · 2h30

Prompt & Context Engineering — Stefan Duprey

TODAY

Session 3 — the pivot to measurement

You will be able to

- Use remotes: clone, fork, push, pull
- Build an eval set and judge outputs
- Test prompt sensitivity & regressions
- Deliver your eval through a pull request

RUN OF SHOW

0:00	Recap
0:10	Git — remotes, fork, GitHub flow
0:50	Evaluation, judging, sensitivity
1:20	Big lab — build an eval harness

• GIT

Remotes & GitHub

1

clone

Copy a remote repository to your machine.

2

push

Send your local commits up to the remote.

3

pull

Fetch and merge everyone else's work.

4

origin

The default name for your main remote.

• GIT

Fork & pull-request workflow

- A fork is your own copy of a repo on GitHub
- Clone your fork, branch, commit, and push to it
- Open a pull request to propose your changes back
- This is how you'll hand in every lab

TODAY

You'll fork the course repo and push your first prompt-engineering lab through it.

WHY

Why evaluation is the whole game

This is the session that turns everything before it into engineering, so let's be clear on why measuring matters this much.

- **'It looks good' doesn't survive change**

The moment you tweak a prompt or swap a model, your gut feeling resets. A held-out eval set doesn't.

- **Without measurement, iteration is guessing**

You change something, it feels better, you move on — and you have no idea if you helped or hurt. A number ends the argument.

- **This is the line between craft and engineering**

Anyone can fiddle with wording. Engineers define success, measure against it, and only keep changes that move the number.

THE PIVOT

From here on, no prompt change ships without a number behind it. That single rule is what this whole course is teaching.

METHOD

Build an eval set

An eval set sounds heavy, but it's just a handful of cases with known-good answers — here's how to put one together.

1

Collect cases

Gather real inputs — and especially the ones that broke before. Ten good cases beat a hundred trivial ones.

2

Define success

For each case, an expected answer or a checkable criterion. If you can't say what 'right' is, you can't evaluate it.

3

Run & score

Execute the prompt across every case and record pass/fail. The course eval harness does this in a few lines.

4

Track over time

Re-run on every change. Your score becomes a scoreboard you can defend to yourself and to a client.

SCORING

LLM-as-judge: grading open-ended output

When there's no single right answer to check against, you can hand the grading to another model — carefully.

- **Use a model to grade a model**

For answers with no single right string (summaries, explanations), a second model applies a rubric and returns a score.

- **Give it a rubric and a strict format**

Score 1–5 on a named quality, return JSON. Vague judges give vague scores.

- **Validate the judge first**

Spot-check it against your own labels before trusting it. A judge you haven't checked is just another opinion.

judge prompt

```
Score the answer 1-5 for
faithfulness to the context.
Return JSON:
{ score: int,
  reason: str }
# rubric in, structured score out
```

DISCIPLINE

Regression & versioning prompts

Prompts are code, so treat them like it: version them, and never assume a fix didn't break something else.

- **Treat prompts like code**

Version them, and pin behaviour with tests. A prompt is a program; your eval set is its test suite.

- **One fix can break another case**

Improving the tricky example often quietly regresses an easy one. Only the full eval set catches this.

- **Keep a scoreboard of versions**

Record each prompt version and its score. You want to see the trend, and be able to roll back.

HABIT

Re-run the whole eval set before you ship any change. If you only remember one sentence from this course, that's the one.

UNCOMFORTABLE TRUTH

Prompts are fragile

Here's the humbling part — tiny, meaningless-looking changes can swing your results, so test for it on purpose.

- **Trivial changes flip outputs**

  Reordering few-shot examples, changing 'Answer:' to 'answer:', an extra blank line — any of these can move results.

- **Whitespace and punctuation matter**

  The model is sensitive to surface form in ways that feel arbitrary. Your parsing has to be robust to it.

- **Temperature adds run-to-run noise**

  Above 0, the same prompt gives different answers. Pin temperature to 0 when you evaluate, or average several runs.

SO WHAT

Test sensitivity on purpose. A prompt that only works by luck will fail the moment anything shifts in production.

THE LOOP

Iterate — measure — refine

Put it all together and you get the loop you'll run for the rest of the course.

1

Baseline

Write the simplest prompt that could work and score it. Now you have a number to beat.

2

Change one thing

One example, one constraint, one reorder — never several at once, or you won't know what helped.

3

Measure

Re-run the eval set and compare the number, not your gut. Keep the evidence.

4

Keep or revert

Keep the change only if it improved the score. Commit the winning version with the numbers in the message.

HANDS-ON LAB

Lab — Build an eval harness

LOCAL · OLLAMA

`prompt-engineering-course/03_evaluation`

TASKS

1. Fork the repo; wrap your S2 classifier in the eval harness
2. Write 15 labelled cases, deliberately including hard ones
3. Score a prompt change and record before/after accuracy
4. Push your branch and open a pull request to hand it in

DELIVERABLE

A runnable eval harness, a scored prompt change, and a PR whose description reports the before/after numbers.

WRAP-UP

Recap — Session 3

- ✓ Measurement is what makes prompting engineering
- ✓ Eval sets + a judge turn opinions into numbers
- ✓ Re-run the whole set to catch regressions
- ✓ Prompts are fragile — test sensitivity deliberately

NEXT SESSION

Reasoning & structured outputs

Make models reason on purpose — then use your eval set to check whether the reasoning actually helped.